

Figure 9.12 A brush with SVG circles as handles

Notice that the circles are also listening for the brush resizing event (as you can tell because the `ew-resize` cursor appears on mouseover). The event listener is assigned to the `g.resize` element, and any SVG elements added to that `g` (such as the circles we just appended) also fire that event. So our handles are not only attractive, they're also functional. Well, at least they're functional.

The next thing we'll do to improve the usability of our brush is to add an indication of where tweets fall on our timeline. We can accomplish this easily with the code in the following listing. The results are shown in figure 9.13.

Listing 9.16 Tweets on a timeline

```
var tweets = d3.selectAll("div.datarow").data();

d3.select("#brushG")
    .selectAll("circle.timeline")
    .data(tweets)
    .enter()
    .append("circle")
    .style("fill","red").style("stroke", "black")
    .style("stroke-width", "1px")
    .style("pointer-events","none")
    .attr("class","timeline").attr("r", 5).attr("cy", 25)
    .attr("cx", function(d) {return timeScale(new Date(d.timestamp))})
```

If our data is still exposed elsewhere, we could just use that.

We don't want the circles to interrupt brush interaction.

Places the circles at their timestamp scaled location

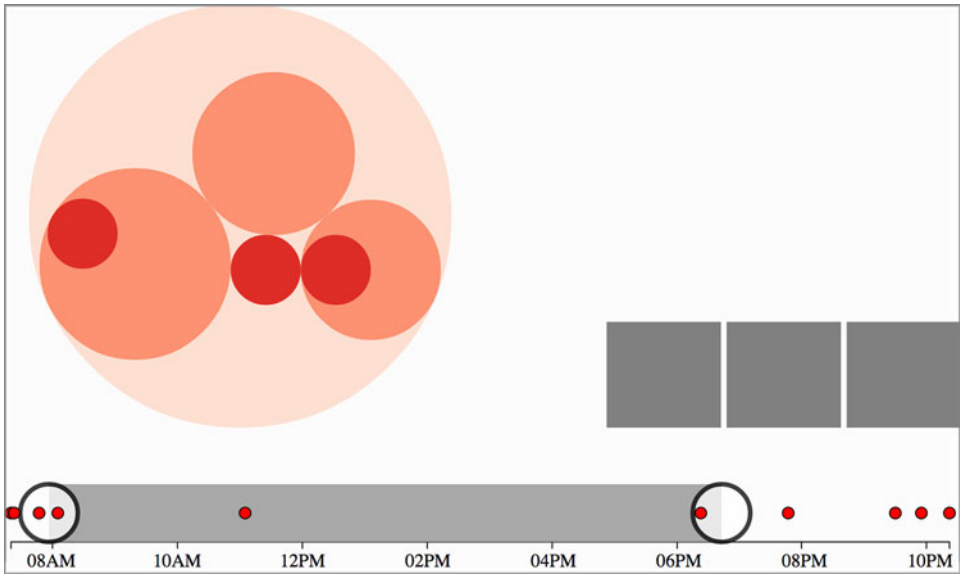


Figure 9.13 A brush with circles appended indicating when tweets occurred